

EduNexus

Professeur IA local pour l'apprentissage personnalisé

Rapport de présentation du projet

Stage — Année universitaire 2025–2026

Étudiant

Nganso Takouteu Thierry

Date

Août 2026

Table des matières

1	Introduction	2
2	Contexte et problématique	2
2.1	Le constat de départ	2
2.2	Les contraintes techniques	2
2.3	La promesse produit	2
3	Architecture technique	2
3.1	Principes fondateurs	2
3.2	Structure du code	3
3.3	Fournisseurs modélaires	3
4	Fonctionnalités principales	3
4.1	Import et indexation	3
4.2	Recherche hybride	3
4.3	Tutorat socratique et citations	3
4.4	Diagnostic et parcours adaptatif	4
4.5	Exercices, quiz et mode épreuve	4
4.6	Répétition espacée et progression	4
4.7	Gamification	4
5	Développement et méthodologie	4
5.1	Approche itérative	4
5.2	Qualité logicielle	4
5.3	Analyse et amélioration	4
6	Perspectives	5
7	Conclusion	5

Introduction

Apprendre à son propre rythme, sans dépendre d'une connexion Internet permanente ni craindre que ses données de cours partent on ne sait où : c'est le problème auquel EduNexus tente de répondre.

Ce projet est né d'une constatation simple. Les plateformes d'apprentissage en ligne offrent des fonctionnalités remarquables, mais elles reposent presque toutes sur le cloud. Pour un étudiant travaillant sur une machine modeste, avec des documents volumineux et une connexion intermittente, cette dépendance devient un obstacle concret. EduNexus propose une alternative **entièrement locale** : un tuteur intelligent qui fonctionne sur votre ordinateur, qui lit vos propres PDF et livres, et qui construit un programme de révision personnalisé sans jamais envoyer un seul octet vers l'extérieur.

Le résultat est une application web autonome, alimentée par des modèles de langage open source, capable de transformer n'importe quel document de cours en un accompagnement pédagogique structuré : diagnostic des lacunes, exercices adaptatifs, fiches de révision et suivi de progression.

Contexte et problématique

Le constat de départ

Les étudiants disposent de deux options extrêmes. D'un côté, les plateformes comme Coursera ou Khan Academy offrent un contenu de qualité, mais imposent un accès à Internet et ne permettent pas toujours d'importer ses propres documents. De l'autre, les outils d'IA grand public répondent de manière générique, sans lien avec le corpus spécifique de l'apprenant. Entre les deux, il manque un outil qui combine la puissance du RAG avec la confidentialité d'un fonctionnement strictement local.

Les contraintes techniques

Le projet cible les machines modestes — un i5 de 7 génération avec 8 Go de RAM. Cette contrainte a guidé chaque décision : modèles GGUF quantifiés gérés séquentiellement, un seul serveur `llama-server` à la fois, embeddings calculés parallèlement en nombre limité, et interface web légère en HTML/CSS/JavaScript vanilla.

La promesse produit

ñ Un professeur IA local qui transforme vos propres documents en un programme de révision vérifiable et personnalisé. ž

Architecture technique

Principes fondateurs

L'architecture repose sur six principes formulés dans une *constitution* du projet (v1.0.0) :

1. **Cur découplé** — Le noyau pédagogique (`tutor/`) ne contient aucun import des bibliothèques d'interface.
2. **Préservation** — Aucune donnée n'est perdue. Livres, conversations et progressions sont persistés dans SQLite.
3. **Tests hors-ligne** — 391 tests s'exécutent sans appel réseau, en mockant les réponses du modèle.

4. **Sécurité locale** — Le serveur ne lie que 127.0.0.1. Pas de donnée envoyée vers l'extérieur.
5. **Légèreté** — Pas de dépendance runtime lourde. NumPy et les outils Python standards suffisent.
6. **Observabilité** — Erreurs journalisées dans `~/.config/ollama-tui/errors.log`.

Structure du code

Le projet suit un agencement **src-layout** avec Python 3.12. Le noyau pédagogique (`tutor/`) contient les services (`service.py`), le stockage (`store.py`), la recherche (`retrieval.py`), les prompts (`prompts.py`), l'évaluation (`assessment.py`), le reranking (`reranker.py`), les extracteurs (`extractors.py`) et les fournisseurs IA (`providers/`). La couche web (`web/`) est un transport fin : `server.py` (FastAPI) délègue tout à `TutorService`, et `tutor.html` est une SPA vanilla.

La séparation est stricte : le noyau pédagogique ne contient aucun import web, garantissant sa réutilisabilité dans tout contexte.

Fournisseurs modélaires

Fournisseur	Usage	Caractéristiques
Ollama	Par défaut	Aucune configuration si Ollama tourne
GGUF local	llama.cpp	Modèles quantifiés, <code>-parallel N</code> pour embeddings
OpenAI-compatible	vLLM, LM Studio	API compatible avec tout provider distant

Les modèles sont chargés séquentiellement — un seul à la fois — pour respecter le budget RAM.

Fonctionnalités principales

Import et indexation

Le pipeline accepte PDF, EPUB, DOCX, PPTX, TXT et MD. Pour chaque document, le texte est extrait, découpé en chunks structurés (frontières de paragraphes, titres de section), transformé en embeddings, et stocké dans SQLite. La déduplication empêche l'indexation en double : un hash du fichier est calculé, et un ré-import détecte automatiquement doublons et mises à jour.

Recherche hybride

La recherche combine **BM25** (lexicale) et **similarité cosinus** (sémantique), fusionnées par *Reciprocal Rank Fusion*. Un reranking post-recherche optionnel ($0,7 \times \text{cosinus} + 0,3 \times \text{Jaccard lexical}$) améliore la précision.

Tutorat socratique et citations

Lorsqu'un étudiant pose une question, le système extrait les passages pertinents et génère une réponse en mode socratique — indices progressifs plutôt que réponses directes. Chaque réponse est accompagnée de citations [Livre X, page Y]. Un système de validation à trois niveaux vérifie leur fondement :

Niveau	Comportement
Citation valide	Affichage normal
Citation faible	Avertissement visible
Réponse non fondée	í Réponse non vérifiée par vos documents ž

Diagnostic et parcours adaptatif

Un quiz de positionnement adapte la difficulté aux réponses. Le rapport identifie forces et faiblesses, puis suggère un parcours ordonné par dépendance pédagogique.

Exercices, quiz et mode épreuve

Le module d’entraînement propose des exercices à difficulté choisie, des quiz à correction immédiate, et un mode í épreuve ž qui importe un sujet d’examen, en extrait les questions, et les résout avec assistance RAG et indices progressifs.

Répétition espacée et progression

Chaque réponse est associée à un concept. Le système calcule un score de maîtrise et programme des sessions de révision au moment optimal. Les erreurs sont enregistrées en détail et consultables par matière.

Gamification

Points d’expérience (+20 quiz, +15 exercice, +10 streak), streak quotidien et badges encouragent la régularité sans détourner de l’apprentissage.

Développement et méthodologie

Approche itérative

Chaque fonctionnalité majeure a fait l’objet d’un document de spécification contenant analyse, plan et critères de validation. Trois itérations principales :

- **Feature 005** — Interface multi-espaces et bibliothèque hiérarchique (36 tâches)
- **Feature 006** — Apprentissage adaptatif et classification (47 tests)
- **Feature 007** — Audit complet et évolution du MVP (95 tâches, 18 phases)

Qualité logicielle

La base de tests compte **391 tests** s’exécutant en moins de 10 secondes, tous hors-ligne via `httpx.MockTransport`. Le noyau pédagogique est testé indépendamment de l’interface web.

Analyse et amélioration

Une analyse externe (Manus AI) a identifié les axes prioritaires. Les recommandations les plus impactantes ont été implémentées : indexation nocturne, file d’attente persistante avec reprise, détection de doublons, maintenance et backup SQLite, optimisations de performance, et refonte responsive de l’interface.

Indicateur	Description	Valeur
Lignes de code	Source principal	$\approx 8\,500$
Tests	Unitaires, contractuels, intégration	391
Fonctionnalités	Espaces, modes, outils	20
Formats supportés	PDF, EPUB, DOCX, PPTX, TXT, MD	6
Fournisseurs	Ollama, GGUF, OpenAI-compatibles	3

Perspectives

1. **Modèle de maîtrise formel** — Entités `Concept`, `MasteryState` et `ReviewSchedule` pour un calcul de progression transparent.
2. **Évaluation RAG** — Corpus de test manuel pour mesurer citations, rappel des chunks et taux de réponses non fondées.
3. **OCR local** — Moteur vrai OCR pour images et PDF scannés, ou périmètre explicitement réduit.
4. **Export et migration** — Export complet de la base et migration du modèle d’embedding sans perte.
5. **Mode *n* local uniquement *z*** — Interrupteur bloquant tout endpoint distant.

Conclusion

EduNexus n’est pas un simple chatbot éducatif. C’est un système d’apprentissage complet qui fonctionne entièrement sur votre ordinateur, comprend vos documents de cours et construit un programme de révision adapté à votre niveau. La confidentialité n’est pas un gadget marketing : c’est un principe architectural.

Le projet a nécessité la maîtrise de domaines variés — RAG, embeddings, streaming SSE, interfaces web autonomes, tests hors-ligne — tout en gardant à l’esprit la contrainte centrale : les machines modestes. Chaque fonctionnalité a été pensée pour fonctionner sur un simple laptop sans GPU.

Aujourd’hui, le système est fonctionnel et testé. Les prochaines étapes porteront sur la profondeur pédagogique (modèle de maîtrise, répétition espacée) et la qualité mesurable (évaluation RAG, corpus de référence). L’objectif reste le même : transformer n’importe quel document de cours en un compagnon d’apprentissage fiable, local et personnalisé.